

Name bhavya rana

Sapid 590034689

Batch 15

Algorithm

1. **Start** the program.
2. Declare two character arrays to store the paragraphs.
3. Call `readParagraph ( )` twice to read Paragraph 1 and Paragraph 2 using `fgets ( )`.
4. Repeatedly display the menu until the user chooses Exit.
5. For **longest and shortest words**, copy the paragraph into a temporary string, tokenize it using `strtok ( )`, compare word lengths using `strlen ( )`, and copy the required words using `strcpy ( )`.
6. For **removing repeated spaces**, scan each character. If a whitespace character occurs immediately after another whitespace character, skip it; otherwise copy it to the output string.
7. For **title case**, scan each character. Convert the first alphabetic character of each whitespace-separated word to uppercase and the remaining alphabetic characters to lowercase.
8. For **word frequencies**, tokenize the paragraph, convert every word to lowercase, compare it with previously stored words using `strcmp ( )`, and increment its frequency or store it as a new distinct word.
9. Repeat the menu after completing each operation.
10. When the user selects **6**, terminate the program.
11. **Stop**.

Test-Case Table

Test	Input / Condition	Operation	Expected Result
1	C programming is very interesting	Longest/shortest	Longest = programming, shortest = C
2	Hello World C	Remove	Hello World C
3	hello WORLD from c	Title case	Hello World From C
4	C is good and c is easy	Word frequency	c=2, is=2, others = 1
5	Hello hello HELLO	Word frequency	hello = 3 because comparison is case-insensitive
6	Programming	Longest/shortest	Both longest and shortest = Programming
7	Empty paragraph	Longest/shortest	No words found.

8	C Programming	Remove	C Programming
9	Choice 0 or 7	Menu	Display invalid-choice message
10	Choice 6	Exit	Program terminates

C program

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX_SIZE 1000
#define MAX_WORDS 200
#define MAX_WORD_LENGTH 100

/* Function prototypes */
void readParagraph(char paragraph[], size_t size);

void findLongestAndShortestWords(const char paragraph[],
                                char longestWord[],
                                char shortestWord[]);

void removeRepeatedSpaces(const char paragraph[],
                          char modifiedParagraph[]);

void convertToTitleCase(const char paragraph[],
                       char titleCaseParagraph[]);

void findWordFrequencies(const char paragraph[]);

/* Function to read a paragraph */
void readParagraph(char paragraph[], size_t size)
{
    if (fgets(paragraph, size, stdin) != NULL)
    {
        paragraph[strcspn(paragraph, "\n")] = '\0';
    }
}

/* Function to find longest and shortest words */
void findLongestAndShortestWords(const char paragraph[],
                                char longestWord[],

```

```

char shortestWord[])
{
    char temp[MAX_SIZE];
    char *word;

    strcpy(temp, paragraph);

    word = strtok(temp, " \t\n.,!?:\"'()[ ]{}");

    if (word == NULL)
    {
        strcpy(longestWord, "");
        strcpy(shortestWord, "");
        return;
    }

    strcpy(longestWord, word);
    strcpy(shortestWord, word);

    while (word != NULL)
    {
        if (strlen(word) > strlen(longestWord))
        {
            strcpy(longestWord, word);
        }

        if (strlen(word) < strlen(shortestWord))
        {
            strcpy(shortestWord, word);
        }

        word = strtok(NULL, " \t\n.,!?:\"'()[ ]{}");
    }
}

```

```

/* Function to remove repeated consecutive spaces */
void removeRepeatedSpaces(const char paragraph[],
                           char modifiedParagraph[])
{
    int i = 0;
    int j = 0;
    int previousWasSpace = 0;

    while (paragraph[i] != '\0')

```

```

{
    if (isspace((unsigned char)paragraph[i]))
    {
        if (!previousWasSpace)
        {
            modifiedParagraph[j++] = ' ';
            previousWasSpace = 1;
        }
    }
    else
    {
        modifiedParagraph[j++] = paragraph[i];
        previousWasSpace = 0;
    }

    i++;
}

modifiedParagraph[j] = '\\0';
}

```

```

/* Function to convert paragraph to title case */
void convertToTitleCase(const char paragraph[],
                        char titleCaseParagraph[])
{
    int i = 0;
    int newWord = 1;

    while (paragraph[i] != '\\0')
    {
        if (isalpha((unsigned char)paragraph[i]))
        {
            if (newWord)
            {
                titleCaseParagraph[i] =
                    toupper((unsigned char)paragraph[i]);

                newWord = 0;
            }
            else
            {
                titleCaseParagraph[i] =
                    tolower((unsigned char)paragraph[i]);
            }
        }
    }
}

```

```

    }
    else
    {
        titleCaseParagraph[i] = paragraph[i];

        if (isspace((unsigned char)paragraph[i]))
        {
            newWord = 1;
        }
    }

    i++;
}

titleCaseParagraph[i] = '\0';
}

/* Function to display frequency of every distinct word */
void findWordFrequencies(const char paragraph[])
{
    char temp[MAX_SIZE];

    char words[MAX_WORDS][MAX_WORD_LENGTH];

    int frequency[MAX_WORDS] = {0};
    int wordCount = 0;

    char *token;
    int i;
    int j;
    int found;

    strcpy(temp, paragraph);

    token = strtok(temp, " \t\n.,!?:\\"'()[ ]{}");

    while (token != NULL)
    {
        /* Convert current word to lowercase */
        for (i = 0; token[i] != '\0'; i++)
        {
            token[i] = tolower((unsigned char)token[i]);
        }
    }
}

```

```

    found = 0;

    /* Search whether word is already stored */
    for (i = 0; i < wordCount; i++)
    {
        if (strcmp(words[i], token) == 0)
        {
            frequency[i]++;
            found = 1;
            break;
        }
    }

    /* New distinct word */
    if (!found && wordCount < MAX_WORDS)
    {
        strcpy(words[wordCount], token);
        frequency[wordCount] = 1;
        wordCount++;
    }

    token = strtok(NULL, " \\t\\n.,!?:\\\"'()[ ]{}");
}

if (wordCount == 0)
{
    printf("No words found.\\n");
    return;
}

printf("%-25s %s\\n", "Word", "Frequency");
printf("-----\\n");

for (j = 0; j < wordCount; j++)
{
    printf("%-25s %d\\n", words[j], frequency[j]);
}
}

/* Main function */
int main(void)
{
    char paragraph1[MAX_SIZE];
    char paragraph2[MAX_SIZE];

```

```

char longest1[MAX_WORD_LENGTH];
char shortest1[MAX_WORD_LENGTH];

char longest2[MAX_WORD_LENGTH];
char shortest2[MAX_WORD_LENGTH];

char modified1[MAX_SIZE];
char modified2[MAX_SIZE];

char title1[MAX_SIZE];
char title2[MAX_SIZE];

int choice;

printf("Enter Paragraph 1:\n");
readParagraph(paragraph1, MAX_SIZE);

printf("\nEnter Paragraph 2:\n");
readParagraph(paragraph2, MAX_SIZE);

do
{
    printf("\n===== MENU =====\n");
    printf("1. Find longest and shortest words\n");
    printf("2. Remove repeated consecutive spaces\n");
    printf("3. Convert paragraphs to title case\n");
    printf("4. Find frequency of every distinct word\n");
    printf("5. Display original paragraphs\n");
    printf("6. Exit\n");

    printf("=====\n");
    printf("Enter your choice: ");

    if (scanf("%d", &choice) != 1)
    {
        printf("Invalid input. Please enter a number.\n");

        while (getchar() != '\n')
            ;

        continue;
    }
}

```

```

switch (choice)
{
    case 1:

        findLongestAndShortestWords(
            paragraph1, longest1, shortest1);

        findLongestAndShortestWords(
            paragraph2, longest2, shortest2);

        printf("\n--- Paragraph 1 ---\n");

        if (strlen(longest1) == 0)
        {
            printf("No words found.\n");
        }
        else
        {
            printf("Longest word  : %s\n", longest1);
            printf("Shortest word : %s\n",
shortest1);
        }

        printf("\n--- Paragraph 2 ---\n");

        if (strlen(longest2) == 0)
        {
            printf("No words found.\n");
        }
        else
        {
            printf("Longest word  : %s\n", longest2);
            printf("Shortest word : %s\n",
shortest2);
        }

        break;

    case 2:

        removeRepeatedSpaces(paragraph1, modified1);
        removeRepeatedSpaces(paragraph2, modified2);

        printf("\nParagraph 1 after removing "

```

```

        "repeated spaces:\n");
printf("%s\n", modified1);

printf("\nParagraph 2 after removing "
        "repeated spaces:\n");
printf("%s\n", modified2);

break;

```

case 3:

```

convertToTitleCase(paragraph1, title1);
convertToTitleCase(paragraph2, title2);

printf("\nParagraph 1 in Title Case:\n");
printf("%s\n", title1);

printf("\nParagraph 2 in Title Case:\n");
printf("%s\n", title2);

break;

```

case 4:

```

printf("\n--- Word Frequencies: Paragraph 1
---\n");
findWordFrequencies(paragraph1);

printf("\n--- Word Frequencies: Paragraph 2
---\n");
findWordFrequencies(paragraph2);

break;

```

case 5:

```

printf("\nOriginal Paragraph 1:\n");
printf("%s\n", paragraph1);

printf("\nOriginal Paragraph 2:\n");
printf("%s\n", paragraph2);

```

```
        break;

    case 6:

        printf("\nExiting program...\n");
        break;

    default:

        printf("\nInvalid choice. "
               "Please enter a choice from 1 to
6.\n");
    }

    } while (choice != 6);

    return 0;
}
```

Output 1 - longest and shortest word

```

Last login: Sat Oct  3 21:00:40 on console
bhavya@Bhavyas-MacBook-Air ~ % nano day41.c
bhavya@Bhavyas-MacBook-Air ~ % clang day41.c -o day41
bhavya@Bhavyas-MacBook-Air ~ % ./day41
Enter Paragraph 1:
c programming is very intresting and programming is easy

Enter Paragraph 2:
hello WORLD welcome to C programming

===== MENU =====
1. Find longest and shortest words
2. Remove repeated consecutive spaces
3. Convert paragraphs to title case
4. Find frequency of every distinct word
5. Display original paragraphs
6. Exit
=====
Enter your choice: 1

--- Paragraph 1 ---
Longest word : programming
Shortest word : c

--- Paragraph 2 ---
Longest word : programming
Shortest word : C

===== MENU =====
1. Find longest and shortest words
2. Remove repeated consecutive spaces
3. Convert paragraphs to title case
4. Find frequency of every distinct word
5. Display original paragraphs
6. Exit
=====
Enter your choice: 2

Paragraph 1 after removing repeated spaces:
c programming is very intresting and programming is easy

Paragraph 2 after removing repeated spaces:
hello WORLD welcome to C programming

===== MENU =====
1. Find longest and shortest words
2. Remove repeated consecutive spaces
3. Convert paragraphs to title case
4. Find frequency of every distinct word
5. Display original paragraphs
6. Exit
=====

```

Output 2 -word frequency

```

===== MENU =====
1. Find longest and shortest words
2. Remove repeated consecutive spaces
3. Convert paragraphs to title case
4. Find frequency of every distinct word
5. Display original paragraphs
6. Exit
=====
Enter your choice: 4

--- Word Frequencies: Paragraph 1 ---
Word                      Frequency
-----
c                          1
programming                2
is                          2
very                       1
intresting                 1
and                        1
easy                       1

--- Word Frequencies: Paragraph 2 ---
Word                      Frequency
-----
hello                      1
world                     1
welcome                   1
to                         1
c                          1
programming                1

===== MENU =====
1. Find longest and shortest words
2. Remove repeated consecutive spaces
3. Convert paragraphs to title case
4. Find frequency of every distinct word
5. Display original paragraphs
6. Exit
=====
Enter your choice: 5

Original Paragraph 1:
c programming is very intresting and programming is easy

Original Paragraph 2:
hello WORLD welcome to C programming

===== MENU =====
1. Find longest and shortest words
2. Remove repeated consecutive spaces
3. Convert paragraphs to title case
4. Find frequency of every distinct word
5. Display original paragraphs
6. Exit
=====
Enter your choice: 6

Exiting program...
bhavya@Bhavyas-MacBook-Air ~ %

```